```
-- ============================================================
--  T Y P E   C I T Y
--  A Dependent Type Theory primer using city structure
-- ============================================================
--
--  This file collects the Lean 4 examples developed during
--  a conversation about modeling a fictional city using DTT.
--
--  The city hierarchy:
--    Type City  : 16×16 miles  (1 total)
--      Quarter  :  8×8 miles   (4 total)
--      District :  4×4 miles   (16 total)
--      Neighborhood : 2×2 miles (64 total)
--      Cell     :  1×1 mile    (256 total)
--
--  Subway infrastructure:
--    Orange Line (N-S) : 16 miles, 9 stops
--    Blue Line   (E-W) : 16 miles, 9 stops
--    Central Station   : intersection of both lines
--
--  Organization:
--    1. Type Universe
--    2. Simple Inductive Types
--    3. Type Families
--    4. Σ-types (dependent pairs)
--    5. Π-types (dependent functions)
--    6. Prop (propositions as types)
--    7. Type City — full hierarchy
--    8. Subway System
--    9. Address type (full nested Σ-type)
-- ============================================================


-- ============================================================
--  1. TYPE UNIVERSE
--  The city blueprint lives in Type
-- ============================================================

-- Every named entity in the city is a type.
-- Types themselves live in a universe called Type (or Sort 1).
-- Prop is a separate universe for propositions.

-- These checks illustrate the universe hierarchy:
-- #check Type      -- Type : Type 1
-- #check Prop      -- Prop : Type

-- A type annotation is a claim about which universe
-- something inhabits. For example:
--   def Quarter : Type := ...   -- lives in Type 0
-- The city plan is a value of type Type — one level up
-- from the city features themselves.


-- ============================================================
--  2. SIMPLE INDUCTIVE TYPES
--  Enumerating the inhabitants of each level
-- ============================================================

-- The four Quarters of Type City (compass quadrants)
inductive Quarter : Type where
  | NW | NE | SW | SE
deriving Repr, DecidableEq

-- A simple example from the original discussion:
```

```
-- five named districts in a smaller city model
inductive SampleDistrict : Type where
  | North | South | East | West | Central
deriving Repr, DecidableEq

-- These are the five inhabitants (terms) of SampleDistrict.
-- Pattern matching must cover all constructors —
-- Lean rejects any match that misses a case.
def districtCode : SampleDistrict → Nat
  | .North   => 1
  | .South   => 2
  | .East    => 3
  | .West    => 4
  | .Central => 5

-- #check SampleDistrict.North   -- SampleDistrict.North : SampleDistrict
-- #check districtCode           -- districtCode : SampleDistrict → Nat


-- ============================================================
--  3. TYPE FAMILIES
--  Types that vary with values — the core DTT move
-- ============================================================

-- A type FAMILY is a function from values to types.
-- Which streets are valid depends on which district you're in.
-- This is the defining move of dependent type theory:
-- types can vary with values.

-- Street types for SampleDistrict (placeholder types)
inductive NorthStreets  : Type where | NorthMain | NorthPark
inductive SouthStreets  : Type where | SouthMain | SouthBay
inductive EastStreets   : Type where | EastMain  | EastRiver
inductive WestStreets   : Type where | WestMain  | WestBlvd
inductive CentralStreets : Type where | CentralGrand | CentralPlaza

-- The type family: return type CHANGES with the district value
def StreetsIn : SampleDistrict → Type
  | .North   => NorthStreets
  | .South   => SouthStreets
  | .East    => EastStreets
  | .West    => WestStreets
  | .Central => CentralStreets

-- These produce DIFFERENT types:
-- #check StreetsIn .North    -- NorthStreets  : Type
-- #check StreetsIn .Central  -- CentralStreets : Type

-- A function parameterized over the family:
-- s has a DIFFERENT TYPE for each value of d
def streetLabel (d : SampleDistrict) (s : StreetsIn d) : String :=
  match d, s with
  | .North,   .NorthMain    => "North Main Street"
  | .North,   .NorthPark    => "North Park Avenue"
  | .South,   .SouthMain    => "South Main Street"
  | .South,   .SouthBay     => "South Bay Road"
  | .East,    .EastMain     => "East Main Street"
  | .East,    .EastRiver    => "East River Drive"
  | .West,    .WestMain     => "West Main Street"
  | .West,    .WestBlvd     => "West Boulevard"
  | .Central, .CentralGrand => "Central Grand Avenue"
  | .Central, .CentralPlaza => "Central Plaza"


-- ============================================================
```

```
--  4. Σ-TYPES (DEPENDENT PAIRS)
--  An address bundles a district with a street IN that district
-- ============================================================

-- A Σ-type bundles a value with something whose type
-- depends on that value. The city address is the canonical
-- example: a district PLUS a street whose type is determined
-- by that specific district.

-- The second component's type CONTAINS the first component's
-- value — that is the dependency.

def SampleAddress : Type :=
  Σ (d : SampleDistrict), StreetsIn d

-- Constructing an address (angle-bracket notation):
def addr1 : SampleAddress := ⟨.Central, .CentralGrand⟩
def addr2 : SampleAddress := ⟨.North,   .NorthPark⟩

-- The projections reveal the dependency:
-- addr1.1 : SampleDistrict
-- addr1.2 : StreetsIn addr1.1   ← type CONTAINS a value

-- #check addr1.1   -- SampleDistrict
-- #check addr1.2   -- StreetsIn SampleDistrict.Central

-- You cannot form ⟨.Central, .NorthMain⟩ — it is a type error.
-- NorthMain does not inhabit CentralStreets.
-- The type system enforces that streets live in their districts.


-- ============================================================
--  5. Π-TYPES (DEPENDENT FUNCTIONS)
--  For every district, a function that knows the right type
-- ============================================================

-- A Π-type is a dependent function: for every value d,
-- produce something whose type may depend on d.
--
-- A city ordinance mandating "each district shall have a
-- designated central plaza" is a Π-type.

def centralPlaza : Π (d : SampleDistrict), StreetsIn d :=
  fun d => match d with
    | .North   => .NorthPark
    | .South   => .SouthMain
    | .East    => .EastMain
    | .West    => .WestMain
    | .Central => .CentralPlaza

-- THE KEY INSIGHT: Π and → are the SAME thing.
-- When the return type doesn't depend on d:
--   Π (_ : SampleDistrict), Nat  ≡  SampleDistrict → Nat
-- So every function type is secretly a Π-type.

-- #check (SampleDistrict → Nat)
-- is Π (_ : SampleDistrict), Nat

-- Non-dependent example (ordinary function):
def districtPopulation : SampleDistrict → Nat
  | .North   => 45000
  | .South   => 38000
  | .East    => 52000
  | .West    => 41000
  | .Central => 28000
```

```
-- ============================================================
--  6. PROP — PROPOSITIONS AS TYPES
--  City facts are propositions; proofs are their inhabitants
-- ============================================================

-- Prop is the universe of propositions.
-- Terms of a Prop type are proofs.
-- This is the Curry-Howard correspondence:
--   propositions are types, proofs are programs,
--   theorem and def are mechanically identical.

-- Adjacency is a Prop (terms of this type are proofs)
def Adjacent : SampleDistrict → SampleDistrict → Prop
  | .North, .Central => True
  | .South, .Central => True
  | .East,  .Central => True
  | .West,  .Central => True
  | .Central, .North => True
  | .Central, .South => True
  | .Central, .East  => True
  | .Central, .West  => True
  | _, _             => False

-- A proof is a term of the proposition type
theorem north_adj_central : Adjacent .North .Central := by
  simp [Adjacent]

theorem south_adj_central : Adjacent .South .Central := by
  simp [Adjacent]

-- A universally quantified claim (Π over Prop):
-- Every district is adjacent to Central
theorem all_adj_central :
    ∀ d : SampleDistrict, d ≠ .Central → Adjacent d .Central := by
  intro d hne
  match d with
  | .North   => simp [Adjacent]
  | .South   => simp [Adjacent]
  | .East    => simp [Adjacent]
  | .West    => simp [Adjacent]
  | .Central => exact absurd rfl hne

-- theorem and def are identical at the kernel level.
-- Prop lives in a special universe with proof irrelevance:
-- all proofs of P are equal (they are interchangeable).


-- ============================================================
--  7. TYPE CITY — THE FULL HIERARCHY
--  The complete four-level structure
-- ============================================================

--  Type City  : 16×16 miles  (1)
--    Quarter  :  8×8 miles   (4)
--    District :  4×4 miles   (16, 4 per Quarter)
--    Neighborhood : 2×2 miles (64, 4 per District)
--    Cell     :  1×1 mile    (256, 4 per Neighborhood)

-- Level 1: Four Quarters (already defined above)
-- Quarter : NW | NE | SW | SE

-- Level 2: Four Districts per Quarter
-- Named by their position within the Quarter
```

```
inductive DistrictPos : Type where
  | A | B | C | D   -- four positions, unnamed for now
deriving Repr, DecidableEq

-- A District is identified by its Quarter + position
-- (type family: which districts exist in a given Quarter)
-- For a symmetric city, all Quarters have the same structure
def DistrictIn (_ : Quarter) : Type := DistrictPos

-- Level 3: Four Neighborhoods per District
inductive NeighborhoodPos : Type where
  | A | B | C | D
deriving Repr, DecidableEq

def NeighborhoodIn (_ : Quarter) (_ : DistrictPos) : Type :=
  NeighborhoodPos

-- Level 4: Four Cells per Neighborhood
inductive CellPos : Type where
  | A | B | C | D
deriving Repr, DecidableEq

def CellIn (_ : Quarter) (_ : DistrictPos) (_ : NeighborhoodPos) : Type :=
  CellPos

-- Counting theorems
-- Each Quarter contains 4 Districts
-- Each District contains 4 Neighborhoods
-- Each Neighborhood contains 4 Cells
-- Total cells: 4 × 4 × 4 × 4 = 256

def totalCells : Nat := 4 * 4 * 4 * 4   -- = 256
#eval totalCells   -- 256


-- ============================================================
--  8. SUBWAY SYSTEM
--  Two lines, 9 stops each, crossing at Central Station
-- ============================================================

--  Orange Line (N-S): 16 miles, stop every 2 miles
--  Blue Line   (E-W): 16 miles, stop every 2 miles
--  Every stop lands on a meaningful hierarchy boundary.
--
--  Mile 0  — terminus
--  Mile 2  — Neighborhood boundary
--  Mile 4  — District boundary (Quarter edge)
--  Mile 6  — Neighborhood boundary
--  Mile 8  — CENTER (the intersection)
--  Mile 10 — Neighborhood boundary
--  Mile 12 — District boundary (Quarter edge)
--  Mile 14 — Neighborhood boundary
--  Mile 16 — terminus

inductive OrangeLineStation : Type where
  | S_Terminus           -- mile 0  (South)
  | S_Neighborhood       -- mile 2
  | S_District           -- mile 4
  | S_InnerNeighborhood  -- mile 6
  | Central              -- mile 8  (THE HUB)
  | N_InnerNeighborhood  -- mile 10
  | N_District           -- mile 12
  | N_Neighborhood       -- mile 14
  | N_Terminus           -- mile 16 (North)
deriving Repr, DecidableEq
```

```
inductive BlueLineStation : Type where
  | W_Terminus           -- mile 0  (West)
  | W_Neighborhood       -- mile 2
  | W_District           -- mile 4
  | W_InnerNeighborhood  -- mile 6
  | Central              -- mile 8  (THE HUB)
  | E_InnerNeighborhood  -- mile 10
  | E_District           -- mile 12
  | E_Neighborhood       -- mile 14
  | E_Terminus           -- mile 16 (East)
deriving Repr, DecidableEq

-- The two lines are structurally isomorphic
-- (same number of stops, same boundary structure)
def orangeToBlue : OrangeLineStation → BlueLineStation
  | .S_Terminus          => .W_Terminus
  | .S_Neighborhood      => .W_Neighborhood
  | .S_District          => .W_District
  | .S_InnerNeighborhood => .W_InnerNeighborhood
  | .Central             => .Central
  | .N_InnerNeighborhood => .E_InnerNeighborhood
  | .N_District          => .E_District
  | .N_Neighborhood      => .E_Neighborhood
  | .N_Terminus          => .E_Terminus

def blueToOrange : BlueLineStation → OrangeLineStation
  | .W_Terminus          => .S_Terminus
  | .W_Neighborhood      => .S_Neighborhood
  | .W_District          => .S_District
  | .W_InnerNeighborhood => .S_InnerNeighborhood
  | .Central             => .Central
  | .E_InnerNeighborhood => .N_InnerNeighborhood
  | .E_District          => .N_District
  | .E_Neighborhood      => .N_Neighborhood
  | .E_Terminus          => .N_Terminus

-- The center station is the unique point belonging to both lines
-- In type theory: the product type OrangeLineStation × BlueLineStation
def centralHub : OrangeLineStation × BlueLineStation :=
  (.Central, .Central)

-- Theorem: the lines have the same number of stops (9)
-- (Both types have exactly 9 constructors — a compile-time fact)


-- ============================================================
--  9. FULL ADDRESS TYPE
--  The complete nested Σ-type for a Type City address
-- ============================================================

-- A full address in Type City:
--   a Quarter,
--   plus a District within that Quarter,
--   plus a Neighborhood within that District,
--   plus a Cell within that Neighborhood.
--
-- Each level's TYPE depends on the VALUE chosen at the level above.
-- This is Dependent Type Theory expressed as city geography.

def Address : Type :=
  Σ (q : Quarter),
  Σ (d : DistrictIn q),
  Σ (n : NeighborhoodIn q d),
    CellIn q d n
```

```
-- Constructing a sample address
-- (NW Quarter, District A, Neighborhood B, Cell C)
def sampleAddress : Address :=
  ⟨.NW, .A, .B, .C⟩

-- Projections
-- sampleAddress.1             : Quarter
-- sampleAddress.2.1           : DistrictIn sampleAddress.1
-- sampleAddress.2.2.1         : NeighborhoodIn ...
-- sampleAddress.2.2.2         : CellIn ...

-- The address is correct BY CONSTRUCTION.
-- An invalid address (wrong Cell type for its Neighborhood)
-- is not a runtime error — it is a TYPE ERROR.
-- The blueprint is rejected before a single brick is laid.

-- ============================================================
--  COMPILE TIME vs RUNTIME — THE TYPE CITY SUMMARY
-- ============================================================
--
--  COMPILE TIME (everything in this file):
--    - The city has exactly 4 Quarters        (inductive type)
--    - A District's validity depends on       (type family)
--      which Quarter it's in
--    - An Address is well-formed              (nested Σ-type)
--    - Every District has a central plaza     (Π-type)
--    - North is adjacent to Central           (theorem + proof)
--    - Orange Line ≅ Blue Line                (isomorphism)
--
--  RUNTIME (not in this file — not needed):
--    - Which station does a user want?
--    - What time does the next train arrive?
--    - How many passengers are on board?
--
--  Type City is a SPECIFICATION, not an application.
--  It lives almost entirely at compile time.
--  The city doesn't execute — it exists as a well-typed structure.
--
-- ============================================================
--  END OF FILE
-- ============================================================
```